NLP-Based Resume-Job Alignment

Tejaswi Chava | Week 1 Research Foundation

Research question and hypothesis

Does combining semantic similarity with lexical matching improve identification of relevant resume-job qualifications compared with lexical matching alone? My hypothesis is that semantic information may help with differently worded qualifications. The combined method remains to be implemented.

Intended evidence

I will compare lexical and hybrid rankings against predefined relevance grades using Precision@K and NDCG. My current evidence consists of input tables, integrity checks, and initial lexical rankings. These establish feasibility, not comparative superiority.

Research inputs

I created 3 fictional jobs and 9 fictional resumes spanning data analysis, web development, and IT support, giving 27 pairs. Each family includes direct wording, paraphrased wording, and partial qualifications. I assigned grades before scoring: 2 for substantial intended task/tool coverage, 1 for partial coverage, and 0 for no relevant evidence in this toy scenario. These are author-defined judgments, not independent annotations or hiring suitability labels. No personal resumes are used.

Environment and organization

My analysis is in src/week1_resume_job_foundation.ipynb. Dependencies are in src/requirements.txt. The recorded environment uses Python 3.12.8, NumPy 1.26.4, pandas 2.2.3, and scikit-learn 1.6.1. The notebook records versions in results/week_1/environment.json and saves inputs, rankings, and check results under results/week_1/. No separate Python script is required.

Execution instructions

Install dependencies with python -m pip install -r src/requirements.txt from the repository root. Open the notebook in Jupyter from the root or src folder, select the Python kernel, restart it, run every cell, and save. Outputs are overwritten under results/week_1/. No internet connection is needed after dependency installation.

Current accomplishments

I generated and checked inputs, fitted unigram TF-IDF on resumes only, transformed jobs with that vocabulary, and computed cosine similarities. I retained the default tokenizer without stop-word removal and used resume ID to break ties. The vocabulary has 81 terms. All 11 checks passed, covering IDs, text completeness, missing values, pair uniqueness and coverage, references and grades, bounded finite scores, self-similarity, and unknown vocabulary. Top pairs were J1-R1 (0.806630), J2-R4 (0.873329), and J3-R7 (0.879927). J1 ranked irrelevant R6 above partial R3. Semantic matching and comparative metrics have not yet been executed.

Problems and boundaries

The sample is small and constructed; labels simplify qualifications and lack independent validation. Generic words can distort rankings, paraphrases may be missed, and TF-IDF does not reliably interpret negation. Passing checks does not validate relevance judgments. I retain the original scope but implement only the initial lexical workflow this week. These outputs do not establish real-world hiring effectiveness.

Week 2 plan

I will expand and freeze the dataset and rubric, document a pretrained sentence-embedding model, predeclare hybrid weights, and implement Precision@K and NDCG with known-example tests. Both methods will use the same candidates. Any tuning will be separated from evaluation. I will inspect failures as well as improvements and will not change labels to favor my hypothesis.